

DSA - Arrays

In most programming languages, **arrays are designed to store elements of the same data type.**

However, some languages, like **Python and JavaScript, allow arrays to hold mixed data types.**

If a language's arrays are restricted to holding elements of the same data type, **alternative data structures like lists or tuples** can be used to store mixed data types.

DSA - Arrays

In languages like Python and JavaScript, arrays can store different data types:

- **Python:** Lists can hold elements of various data types.
- **JavaScript:** Arrays can hold a mix of primitive types (like strings, numbers, and booleans) and other objects.

In languages like C++ and Java, arrays typically hold elements of the same data type:

- **C++:** Arrays are designed to store elements of a specific type, and it's not possible to have an array that stores both integers and strings, for example.
- **Java:** Arrays can store objects of a particular type, but not different types within the same array.

DSA - Arrays

- An array is a **type of linear data structure** that is defined as a **collection of elements with same or different data types**.
- They exist in both **single dimension and multiple dimensions**.
- These data structures come into picture **when there is a necessity to store multiple elements of similar nature together at one place**.

Memory Address	→	2391	2392	2393	2394	2395
Array Values	→	12	34	68	77	43
Array Index	→	0	1	2	3	4

DSA - Arrays

The difference between an array index and a memory address is that the **array index acts like a key value to label** the elements in the array.

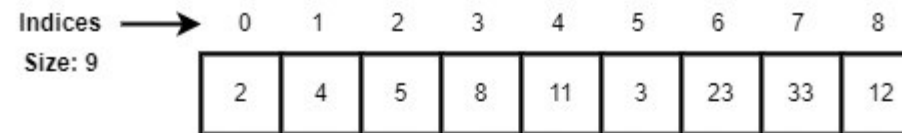
However, a **memory address is the starting address of free memory available.**

Following are the important terms to understand the concept of Array.

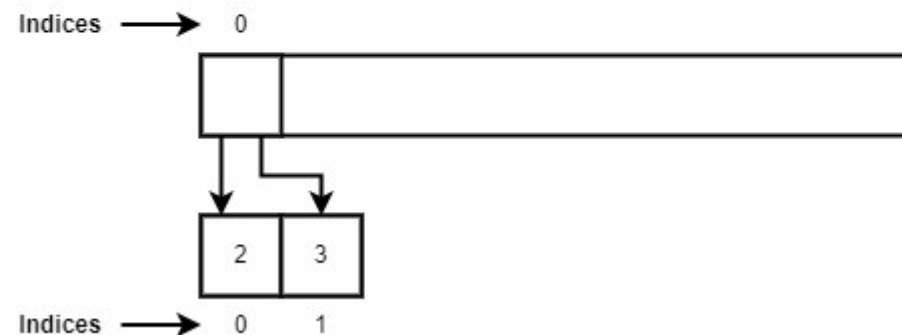
- **Element** – Each item stored in an array is called an element.
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.
- Arrays are represented as a collection of buckets where each bucket stores one element.
- These buckets are indexed from '0' to 'n-1', where n is the size of that particular array.
- For example, an array with size 10 will have buckets indexed from 0 to 9.

DSA - Arrays

- This indexing will be similar for the multidimensional arrays as well.
- If it is a **2-dimensional array**, it will have **sub-buckets** in each bucket.
- Then it will be indexed as `array_name[m][n]`, where `m` and `n` are the sizes of each level in the array.



Single Dimensional Array



Multi Dimensional Array

DSA - Arrays

Creating a 2 Dimensional Array

<https://www.youtube.com/watch?v=sGrY0h2uem0&list=PLGZqdNxqKzfYVbCaAKTPHVjz-VjQtBzbl&index=3>

DSA - Arrays

Basic Operations in Arrays

The basic operations in the Arrays are **insertion, deletion, searching, display, traverse, and update**. These operations are usually performed to either modify the data in the array or to report the status of the array.

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

DSA - Arrays

NumPy Arrays

- NumPy arrays are a part of the NumPy library, which is a powerful tool for numerical computing in Python.
- These arrays are designed for high-performance operations on large volumes of data and support multi-dimensional arrays and matrices.
- This makes them ideal for complex mathematical computations and large-scale data processing.

Note:

- *Choose NumPy arrays for scientific computing, where you need to handle complex operations or work with multi-dimensional data.*
- *Use Python's array module when you need a basic, memory-efficient container for large quantities of uniform data types, especially when your operations are simple and do not require the capabilities of NumPy.*

DSA - Arrays

Example: Create a One-dimensional Array

```
import numpy as np
a = np.array([1, 2, 3])
print(a)
```

Example: Create a Multi-dimensional Array

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(a)
```

DSA - Arrays

Accessing Array – Single Dimensional Arrays

Example: Get the first element from an array

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[0])
```

Example: Get the third and fourth element

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[2] + arr[3])
```

DSA - Arrays

Accessing Array – 2 Dimensional Arrays

Example: Get the First row, Second element

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

Example: Get the Second row, Fifth element

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('5th element on 2nd row: ', arr[1, 4])
```

DSA - Arrays

Creating a 3 Dimensional Array

<https://www.youtube.com/watch?v=SjtXNJy9RCk&list=PLGZqdNxqKzfYVbCaAKTPHVjz-VjQtBzbl&index=4>

DSA - Arrays

Accessing Array – 3 Dimensional Arrays

Example: Access the third element of the second array of the first array

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])
```

Example Output - Ponder

`arr[0, 1, 2]` prints the value 6

DSA - Arrays

Ponder this ...

```
import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(arr[0, 1, 2])
```

What will be the printed result?

DSA - Arrays

Sorting Arrays

Example: Implementing the classical bubble sort algorithm to obtain a sorted array

```
import array as arr
a = arr.array('i', [10,5,15,4,6,20,9])
for i in range(0, len(a)):
    for j in range(i+1, len(a)):
        if(a[i] > a[j]):
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
print (a)
```

DSA - Arrays

Ponder this ...

```
import numpy as np  
a = np.array([10, 5, 15, 4, 6, 20, 9])  
sorted_a = np.sort(a)  
print(sorted_a)
```

DSA – Arrays

Individual Exercise – Due Next week

Activity: Array basic operations

Reference: Using the elements (1,2,3,4,5,6,7,8,9)

Demonstrate: Write the code for the following – hand coding

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at a given index.
- **Deletion** – Deletes an element at a given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.
- **Finding the minimum value.**